$ Aide-mémoire Kubernetes

Les commandes et notions essentielles de Kubernetes, groupées par usage.
Chaque partie d'une commande garde toujours la même couleur pour qu'on la repère d'un coup d'œil.

```
commande racine sous-commande option / flag valeur à remplacer
```

Qu'est-ce que Kubernetes (K8s) ?

Kubernetes est un système open source qui **automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées**. Là où Docker lance des conteneurs sur une machine, Kubernetes les orchestre sur un ensemble de machines (un *cluster*) : il décide où les placer, en relance automatiquement en cas de panne et en ajoute ou en retire selon la charge.

L'abréviation **K8s** vient du nombre de lettres entre le « K » et le « s » de Kubernetes (8). On pilote un cluster avec l'outil en ligne de commande `kubectl`.

En pratique, Kubernetes aide à **lancer et gérer automatiquement de nombreux conteneurs**. Exemple : une application découpée en microservices (`users`, `products`, `orders`, `notifications`), chacun dans son conteneur Docker, à faire tourner sur plusieurs serveurs. Plutôt que de choisir à la main sur quel serveur lancer chaque conteneur, on décrit à Kubernetes ce qu'on veut obtenir, et il répartit lui-même les conteneurs sur les serveurs disponibles.

Ce que Kubernetes apporte

• **Déploiement automatique** (*Automatic Deployment*) : il simplifie le déploiement sur de nombreuses machines.
• **Disponibilité** (*Availability*) : il s'assure que les applications tournent toujours, même en cas de panne. Si un conteneur plante (ex. `users`), Kubernetes le détecte et en relance automatiquement un nouveau.
• **Mise à l'échelle** (*Scalability*) : il ajoute ou retire des instances d'un service selon le trafic. Si la charge augmente sur `products`, il lance des copies supplémentaires de ce conteneur, puis les retire quand la charge baisse.
• **Répartition de charge** (*Load balancing*) : il répartit le trafic entrant entre les différentes instances d'un même service, pour qu'aucune ne soit surchargée.

Un peu d'histoire

Kubernetes a été développé par **Google**, qui l'a rendu **open source en 2014**, puis confié à la **Cloud Native Computing Foundation (CNCF)**, qui assure aujourd'hui sa gouvernance.

Orchestration de conteneurs

L'**orchestration de conteneurs** est le processus qui consiste à **gérer automatiquement de nombreux conteneurs** : les **démarrer**, les **arrêter**, les **redémarrer** et les **mettre à l'échelle** (en ajouter ou en retirer). Kubernetes est l'outil d'orchestration le plus répandu.

Les fonctionnalités d'orchestration de Kubernetes

• **Déploiement automatisé** (*Automated deployment*) : on décrit l'état voulu (quelle image, combien d'instances), Kubernetes s'occupe de le mettre en place.
• **Mise à l'échelle** (*Scaling*) : augmenter ou réduire le nombre d'instances d'un service, manuellement ou selon la charge.
• **Planification** (*Scheduling*) : choisir sur quel serveur placer chaque conteneur, selon les ressources disponibles. Exemple : plusieurs instances de `users` réparties sur différents serveurs, `products` regroupé sur un autre.
• **Auto-réparation** (*Self-healing*) : relancer automatiquement un conteneur qui plante, ou le déplacer si un

serveur tombe en panne.
• **Répartition de charge** (*Load balancing*) : répartir le trafic entre les instances d'un même service.
• **Mises à jour progressives** (*Rolling updates*) : passer à une nouvelle version de l'application en remplaçant les instances une par une, sans interruption de service.
• **Retours arrière** (*Rollbacks*) : revenir rapidement à la version précédente si la nouvelle pose problème.
• **Découverte de services** (*Service Discovery*) : permettre aux services de se trouver par leur nom, sans connaître les adresses IP des conteneurs, qui changent à chaque redémarrage. C'est essentiel dans une **architecture microservices**, où les services s'appellent en permanence : `orders` appelle simplement `http://users/…`, et Kubernetes l'oriente vers une instance disponible de `users`, quel que soit le serveur où elle tourne. Kubernetes intègre ce mécanisme nativement (via ses *Services* et son DNS interne), là où une application Spring Cloud classique utilise un annuaire dédié comme Eureka.

Architecture d'un cluster

Pod

Le **Pod** est la **plus petite unité** que Kubernetes déploie et gère. Il enveloppe un conteneur (parfois plusieurs, très liés), par exemple celui d'une application Spring Boot, et lui fournit :
• un **stockage partagé** (volumes accessibles à tous les conteneurs du Pod) ;
• un **réseau partagé** : une adresse IP propre au Pod, ses conteneurs communiquent entre eux via `localhost` ;
• des **métadonnées** sur l'environnement dans lequel il tourne (nom, labels, namespace…).
On ne lance donc pas directement des conteneurs dans Kubernetes, mais des Pods.

Node (nœud de travail)

Un **Node** est une machine (physique ou virtuelle) du cluster qui **exécute les Pods**. Un cluster en compte généralement plusieurs : si un Node tombe en panne, Kubernetes recrée ses Pods sur un autre Node encore disponible (auto-réparation). Chaque Node fait tourner trois composants :
• **`kubelet`** : l'agent du Node. Il démarre et arrête les conteneurs, vérifie qu'ils restent en bonne santé, leur attache le stockage dont ils ont besoin, et rend compte de ce qui tourne au Control Plane ;
• **`kube-proxy`** : gère les règles réseau du Node pour que le trafic destiné à un service atteigne le bon Pod (base de la découverte de services et de la répartition de charge) ;
• le **runtime** de conteneurs (ex. `containerd`) : le logiciel qui exécute réellement les conteneurs à partir des images.

Control Plane (plan de contrôle)

Le **Control Plane** est un ensemble de **composants de gestion** qui tournent sur une ou plusieurs machines dédiées (souvent trois, pour qu'il reste disponible si l'une tombe). C'est le « cerveau » du cluster : il reçoit les demandes (par exemple via `kubectl`), décide sur quel Node placer chaque Pod, et surveille en permanence que l'état réel correspond à l'état voulu. Il communique en continu avec le `kubelet` de chaque Node.

Cluster

Le **cluster** est l'ensemble formé par le **Control Plane** et les **Nodes de travail** qui exécutent les Pods. C'est à ce cluster que l'on s'adresse avec `kubectl`.